Design and Verification of APB-Based Watchdog Timer using SystemVerilog

U Gnanavi
Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
gnanavi.23ece@cambridge.edu.in

Ullas Dhankoti S
Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
ullas.23ece@cambridge.edu.in

Vindhya D
Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
vindhya.23ece@cambridge.edu.in

V M Swathika
Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
swathika.23ece@cambridge.edu.in

Dr. Indu K
Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
indu.ece@cambridge.edu.in

Mr. Uday Prasad
Design Engineer
Elevium – by Nanochip
External Guide
udayprasad@elevium.ai

***Abstract*—Modern embedded systems and System-on-Chip architectures require reliable monitoring mechanisms to detect failures and ensure uninterrupted operation. Software crashes, deadlocks, infinite loops, and delayed system responses may result in abnormal behavior and reduce system reliability. To address these issues, a watchdog timer is commonly used as a fault detection and recovery mechanism.**

This paper presents the design and verification of an APB-based Watchdog Timer implemented using SystemVerilog. The architecture includes programmable registers, a configurable 32-bit down counter, interrupt generation logic, and reset functionality integrated through an AMBA Advanced Peripheral Bus interface. A two-stage timeout mechanism is implemented where the first timeout generates an interrupt and the second timeout generates a reset signal if the watchdog is not refreshed.

A structured verification environment including generator, driver, monitor, reference model, and scoreboard components is developed to validate functionality. Simulation results confirm successful timeout detection, refresh operation, interrupt generation, and reset behavior.

***Index Terms*—Watchdog Timer, APB Protocol, SystemVerilog, RTL Design, SoC, Functional Verification**

I. Introduction

Embedded systems are used extensively in automotive electronics, industrial systems, communication devices, medical applications, and consumer electronics. As system complexity increases, software and hardware failures become more common. Failures such as deadlocks, infinite loops, and timing violations may force systems into unresponsive conditions.

To improve reliability and provide fault recovery mechanisms, watchdog timers are integrated into embedded systems. A watchdog timer continuously monitors system activity and expects periodic refresh operations from the processor. If the watchdog is not refreshed within a predefined timeout period, the system assumes abnormal operation and initiates corrective actions.

In this work, an APB-based watchdog timer is implemented as a System-on-Chip peripheral. The watchdog includes configurable timeout registers, interrupt generation, reset functionality, and refresh mechanisms integrated with an APB communication interface.

The design is implemented using SystemVerilog and validated using a layered verification environment.

A. Contributions of the Proposed Work

- Design and implementation of APB-based Watchdog Timer architecture.
- Development of configurable timeout and watchdog refresh mechanisms.
- Implementation of interrupt and reset generation logic.
- Development of a layered SystemVerilog verification environment.
- Functional verification using simulation and waveform analysis.
- Evaluation of watchdog operation and timeout behavior.

II. Literature Survey

Watchdog timers are widely used in embedded systems and System-on-Chip architectures to improve system reliability and fault recovery. Several researchers have proposed different

watchdog designs, communication techniques, and verification approaches to ensure proper monitoring of system operation.

In [1], ARM introduced the SP805 watchdog timer architecture using a two-stage timeout mechanism. The proposed architecture generates an interrupt signal during the first timeout event and produces a reset signal if the watchdog is not refreshed before the second timeout event. This approach improves system safety and fault handling capability.

In [2], the AMBA APB protocol was presented as a low-complexity communication protocol intended for peripheral devices in SoC architectures. The protocol supports efficient register-based communication while maintaining simple control logic and reduced hardware overhead.

In [3], a modular RTL implementation methodology using Verilog HDL was discussed. The work highlighted that modular design approaches improve scalability and simplify debugging and future system integration.

In [4], SystemVerilog-based verification methodologies using generators, drivers, monitors, and scoreboards were analyzed. The study showed that structured verification environments improve functional validation and simplify debugging processes.

In [5], a programmable watchdog timer was proposed for embedded applications with configurable timeout support. The design demonstrated flexible operation suitable for different system requirements.

In [6], an FPGA-based watchdog architecture was implemented for industrial applications. Simulation results showed reliable timeout detection and automatic recovery support under abnormal operating conditions.

In [7], APB-based peripheral communication for SoC systems was discussed. The work emphasized memory-mapped communication and standardized interfacing techniques for improved modularity.

In [8], finite state machine based hardware control techniques were analyzed. The implementation provided structured control flow and improved operational efficiency in digital systems.

In [9], watchdog mechanisms for safety-critical applications were studied. The work demonstrated that watchdog timers significantly improve fault tolerance and reduce system downtime.

In [10], layered verification methodologies with reference models and scoreboards were discussed. The use of self-checking environments improved output validation and verification efficiency.

From the literature survey, it is observed that most existing works focus on watchdog functionality or communication protocol implementation independently. Very few studies combine APB communication, watchdog operation, and complete verification architecture into a single framework. Therefore, this work proposes an APB-based Watchdog Timer implemented using SystemVerilog with a structured verification environment..

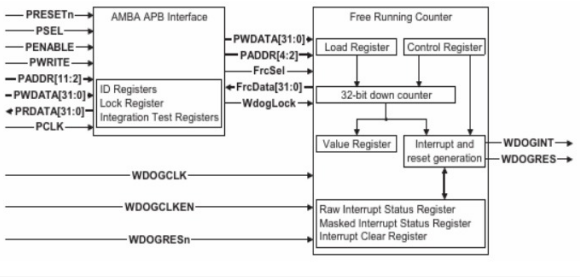

Fig. 1. Block Diagram of APB-Based Watchdog Timer

III. System Architecture

The proposed APB-based Watchdog Timer architecture consists of multiple functional modules integrated together to provide fault monitoring, timeout detection, and system recovery support. The architecture follows a modular design methodology in which each block performs a dedicated operation. Communication between the processor and watchdog timer is achieved using the AMBA Advanced Peripheral Bus (APB) protocol.

The architecture continuously monitors system activity and expects periodic refresh operations from the processor. If refresh signals are not received within a predefined timeout interval, the watchdog initiates interrupt and reset operations.

The major functional blocks of the proposed architecture are described below.

1. APB Interface

The APB interface acts as a communication bridge between the processor and watchdog timer registers. The processor accesses watchdog functionality through memory-mapped register operations. Signals such as PCLK, PSEL, PENABLE, PWRITE, PADDR, and PWDATA are used for communication.

The APB protocol simplifies peripheral communication and reduces hardware complexity while supporting efficient control operations.

2. Load Register

The load register stores the timeout value programmed by the processor. This value initializes the watchdog counter and determines the maximum allowable delay before timeout detection occurs.

The timeout value can be configured according to system requirements, allowing flexibility in watchdog operation.

3. Control Register

The control register enables configuration of watchdog functionality. It controls interrupt enable, reset enable, and watchdog activation operations.

The processor writes control information into this register through APB transactions.

4. Interrupt Clear Register

The interrupt clear register is used to refresh or service the watchdog timer. Writing appropriate values into this register clears interrupt conditions and reloads the watchdog counter.

Periodic refresh operations prevent timeout generation under normal operating conditions.

5. 32-bit Down Counter

The down counter acts as the core component of the watchdog timer. The counter continuously decrements with every clock cycle when watchdog operation is enabled.

Whenever refresh occurs, the counter reloads with the programmed timeout value. Failure to refresh the watchdog causes the counter to eventually reach zero.

6. Interrupt Generation Logic

The watchdog uses a two-stage timeout mechanism. During the first timeout event, an interrupt signal named WDOGINT is generated.

This interrupt provides an early warning indication and allows software to perform corrective actions.

7. Reset Generation Logic

If the watchdog is not refreshed after interrupt generation, a second timeout event occurs and WDOGRES is asserted.

This signal resets the system and restores normal operation after failure detection.

The modular organization of the architecture improves scalability, simplifies debugging, and supports future integration with larger SoC platforms.

IV. Methodology

The proposed APB-based Watchdog Timer is developed using a modular RTL design methodology in SystemVerilog. The complete design process includes requirement analysis, architecture development, RTL implementation, APB integration, verification, and result validation. A structured approach is followed to simplify implementation and improve system reliability.

The methodology is divided into different stages to ensure proper development and verification of the watchdog functionality.

A. Requirement Specification

The first stage involves identifying the functional requirements of the watchdog system. The design should continuously monitor processor activity and detect abnormal system behavior. The watchdog must support timeout generation, interrupt handling, reset functionality, and processor-controlled refresh operations.

The major system requirements are listed below:

- Support configurable timeout values.
- Enable communication using the APB protocol.
- Provide watchdog refresh functionality.
- Generate interrupt signals during timeout events.
- Generate reset signals during watchdog failure.
- Support programmable register configuration.
- Ensure reliable operation under different conditions.

These requirements form the basis for architecture development and RTL implementation.

B. Architectural Design

After requirement analysis, a block-level architecture is developed. The architecture consists of APB communication modules, control registers, watchdog timer logic, and timeout handling circuitry.

The architecture includes:

- APB Interface
- Load Register
- Control Register
- Interrupt Clear Register
- Watchdog Down Counter
- Interrupt Logic
- Reset Logic

Each block performs a dedicated operation and collectively provides complete watchdog functionality.

C. RTL Implementation

The watchdog timer is implemented using synthesizable SystemVerilog RTL coding techniques. Modular implementation is followed to improve readability and simplify debugging.

The APB interface module receives processor requests and updates watchdog registers accordingly. The timeout value is stored inside programmable registers and loaded into the watchdog counter.

The down counter continuously decrements with each clock cycle when enabled. Whenever a watchdog refresh event occurs, the counter reloads with the programmed timeout value.

If the watchdog is not refreshed within the specified timeout interval, timeout detection logic activates interrupt and reset generation circuitry.

D. APB Communication Process

The APB interface controls communication between processor and watchdog modules.

The communication sequence follows:

- Processor selects watchdog peripheral.
- Address and control signals are transmitted.
- Register write/read operations are performed.
- Watchdog parameters are updated.
- Counter values are monitored continuously.

This communication mechanism simplifies processor interaction with watchdog functionality.

E. Verification Strategy

Functional verification is performed using a structured SystemVerilog verification environment.

The verification environment includes:

- Generator
- Driver
- Monitor
- Reference Model
- Scoreboard

The generator creates APB transactions and watchdog operations. The driver converts transactions into DUT signals. The

monitor observes output behavior and transfers information to the scoreboard.

The reference model predicts expected behavior and compares results against actual DUT outputs.

F. Validation and Output Monitoring

Simulation waveforms are analyzed to verify proper watchdog operation.

The following parameters are validated:

- Timeout detection
- Interrupt generation
- Reset generation
- Refresh operation
- APB register access
- Counter functionality

Waveform analysis confirms protocol correctness and expected signal behavior during different operating conditions.

The modular methodology improves scalability, simplifies future modifications, and supports integration into larger SoC architectures.

V. Flowchart

The flowchart represents the operational sequence of the APB-based Watchdog Timer. The process begins with system initialization and loading of timeout values through APB communication. Once configuration is completed, the watchdog timer starts monitoring system activity continuously.

The processor is required to periodically refresh the watchdog before the timeout interval expires. Under normal operating conditions, the counter reloads continuously and system execution proceeds without interruption.

However, if refresh operations are not received within the specified interval, the watchdog assumes abnormal behavior and initiates timeout handling procedures. Initially, an interrupt signal is generated as a warning indication. If no corrective action is taken, the watchdog subsequently generates a reset signal.

The complete operational flow of the watchdog timer is illustrated in Fig. 2.

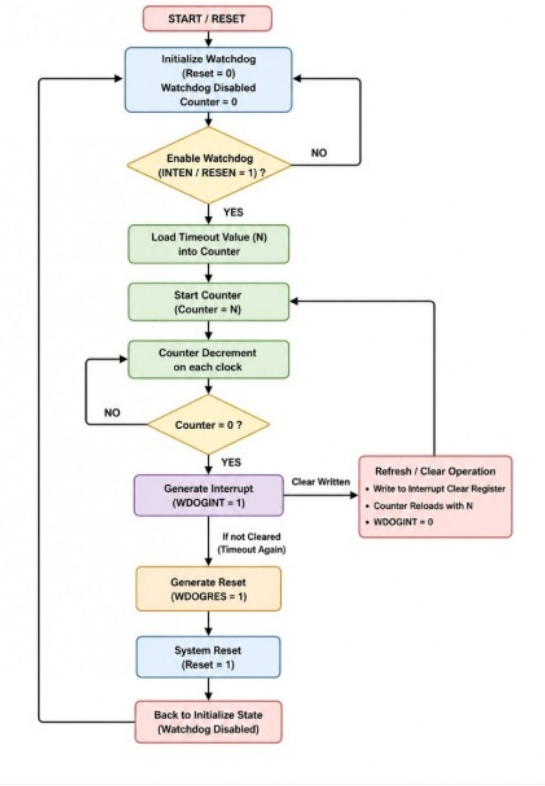

Fig. 2. Flowchart of APB-Based Watchdog Timer operation

A. Flowchart Explanation

a. Start / Initialization

The process begins with system initialization. During this stage, clock signals, reset signals, internal registers, and watchdog modules are initialized into known operating conditions.

b. Configure APB Registers

The processor configures watchdog parameters using APB communication. Timeout values, interrupt enable settings, and control information are written into internal registers.

c. Load Timeout Value

The configured timeout value is loaded into the watchdog down counter. This value determines the maximum duration allowed before timeout detection occurs.

d. Enable Watchdog Operation

After configuration, the watchdog is enabled and starts decrementing the counter continuously.

e. Monitor Refresh Signal

The watchdog continuously monitors refresh signals generated by the processor.

The system checks:

- Whether refresh operation occurred
- Whether timeout interval expired
- Current counter status

f. Refresh Received?

If refresh occurs before timeout expiration, the watchdog counter reloads with the programmed value and normal operation continues.

g. Generate Interrupt Signal

If refresh does not occur and the first timeout event is reached, the watchdog generates an interrupt signal named WDOGINT.

This signal acts as an early warning indication for software recovery.

h. Generate Reset Signal

If refresh is still not received after interrupt generation, the second timeout stage occurs and the watchdog asserts WDOGRES.

This reset signal forces system restart and restores operation.

i. Return to Idle State

After successful refresh or system reset, watchdog operation returns to monitoring mode and waits for subsequent events.

The flowchart clearly describes watchdog functionality and illustrates the sequence of timeout detection, refresh handling, interrupt generation, and system reset operations.

VI. Result Analysis

The proposed APB-based Watchdog Timer was simulated and analyzed to verify its functionality under different operating conditions. Functional verification was performed to validate APB communication, watchdog refresh operation, timeout detection, interrupt generation, and reset behavior.

Initially, APB transactions were used to configure timeout values and control settings. During normal operation, periodic refresh signals were provided before timeout expiration. Under these conditions, the watchdog counter reloaded correctly and no interrupt or reset event occurred.

To verify timeout behavior, refresh operations were intentionally stopped during simulation. As the watchdog counter continued decrementing, the timeout condition was reached and the interrupt signal WDOGINT was generated. If the watchdog remained unrefreshed, the reset signal WDOGRES was asserted during the next timeout event.

The simulation waveforms confirmed proper APB communication and correct watchdog operation. The observed results verified timeout detection, watchdog refresh functionality, interrupt generation, and reset behavior under different test scenarios.

A. Simulation Results

The waveform analysis confirmed:

- Correct APB register operations
- Successful timeout detection
- Proper watchdog refresh functionality
- Correct interrupt generation
- Successful reset operation

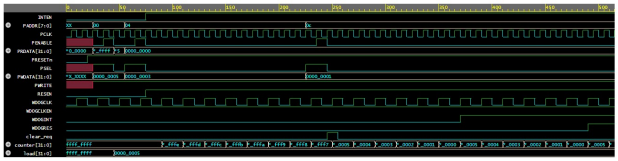

Fig. 3. Simulation waveform showing APB communication and watchdog operation

VII. Conclusion

In this work, an APB-based Watchdog Timer was designed and verified using SystemVerilog for fault monitoring in embedded systems and System-on-Chip architectures. The architecture includes an APB interface, control registers, watchdog counter, and interrupt/reset generation logic.

A two-stage timeout mechanism was implemented where an interrupt signal is generated during the first timeout event and a reset signal is generated if no refresh operation occurs. Functional verification and simulation results confirmed correct APB communication, timeout detection, refresh operation, interrupt generation, and reset behavior.

Overall, the proposed design provides a reliable and efficient solution for improving system monitoring and fault recovery in embedded applications.

VIII. Future Work

The proposed APB-based Watchdog Timer can be further improved by adding advanced features and extending its functionality. Future enhancements may include implementation of a window watchdog mechanism, where refresh operations are allowed only within a specific time interval to improve fault detection accuracy.

The design can also be implemented on FPGA platforms for real-time hardware validation and performance analysis. In addition, support for programmable clock division and dynamic timeout configuration can be included to improve flexibility.

Further improvements may involve adopting UVM-based verification methodologies and integrating the watchdog timer with larger System-on-Chip architectures for practical embedded applications.

References

[1] ARM Limited, "SP805 Watchdog Timer Technical Reference Manual," ARM DDI 0270B, 2007.

[2] ARM Limited, "AMBA 3 APB Protocol Specification," ARM IHI 0024C, 2004.

[3] S. Palnitkar, "Verilog HDL: A Guide to Digital Design and Synthesis," 2nd ed., Pearson Education, 2003.

[4] C. Spear and G. Tumbush, "SystemVerilog for Verification: A Guide to Learning the Testbench Language Features," 2nd ed., Springer, 2012.

[5] Texas Instruments, "MSP430 Ultra-Low-Power Microcontroller User's Guide," Technical Documentation, 2016.

[6] R. Kumar and S. Patel, "FPGA-Based Watchdog Architecture for Industrial Applications," in *Proceedings of IEEE International Conference*, pp. 1–5.

[7] M. S. L. Mukthi and A. R. Aswatha, "Design and Implementation of an Interfacing Protocol Between APB and SoC Components," *International Journal of Scientific Engineering and Science*, 2017.

[8] M. M. Mano and M. D. Ciletti, "Digital Design," 5th ed., Pearson, 2013.

[9] N. H. E. Weste and D. Harris, "CMOS VLSI Design: A Circuits and Systems Perspective," 4th ed., Pearson, 2011.

[10] IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, "IEEE Std 1800-2017."